

SIH26249 — 05 Build Plan & Tasks

SIH 2026 — SIH26249

Air Power: Predictive Maintenance & Fleet Availability

Ministry of Defence — Defence Services Staff College

05 — 36-Hour Build Plan (numbered tasks, top-to-bottom executable)

How to read this: a coding agent executes T1 → T19 in order. Each task lists goal, steps, files, dependencies, definition of done, and the agent-skill it needs. Skill names below are logical roles the parent orchestrator maps to actual coding agents: `data-eng` (Python data/generation), `ml-eng` (Scikit-learn modeling), `backend-eng` (FastAPI), `frontend-eng` (Vanilla JS + Canvas 2D), `qa-eng` (pytest/evidence).

Build order rationale: data → model evidence → decision logic → API → UI → demo. Get the RMSE-vs-baseline printout (T6) as early as possible — it is the headline judge evidence and de-risks the ML story before UI work starts.

PHASE A — DATA FOUNDATION (hours 0–8)

T1. Project scaffold + config

- **Goal:** repo layout, one uvicorn process serving API + static, thresholds config.
- **Steps:** create dirs `gen/` `features/` `models/` `core/` `api/` `web/` `config/` `tests/`; `config/thresholds.yaml` (critical_rul_h=24, forecast_days=7, cann_rate_warn, priority weights); `requirements.txt` (fastapi, uvicorn, scikit-learn, pandas, numpy, pytest, pydantic); `api/main.py` serving `web/` static.
- **Files:** create all above; `README.md` with run instructions.
- **Deps:** none. **Skill:** `backend-eng`.
- **Done:** `uvicorn api.main:app` serves a placeholder page at `/` and `/docs` loads.

T2. SQLite schema

- **Goal:** all 10 tables from `04-data-models-and-logic.md` §4.1.
- **Steps:** `gen/schema.sql`; `gen/db.py` init function; every row-writing path sets the synthetic marker.
- **Files:** `gen/schema.sql`, `gen/db.py`. **Deps:** T1. **Skill:** `data-eng`.
- **Done:** fresh `readyfleet.db` creates all tables; inserting one aircraft + component round-trips.

T3. Synthetic generator v1 (components + sensors)

- **Goal:** degradation curves, sensor streams, RUL labels per §4.2.
- **Steps:** per-component $\text{health}(t)=1-a \cdot \exp(b \cdot t)$ with randomized a,b; 3 op settings + 12 sensors + Gaussian noise; RUL label; 5% infant mortality; censored no-fault units.
- **Files:** `gen/generate.py`, `gen/degradation.py`. **Deps:** T2. **Skill:** `data-eng`.
- **Done:** N tails × M components produce sensor CSV/DB rows; plot of one unit shows clean degradation + noise.

T4. Generator v2 (spares, crew, sorties, records) + determinism

- **Goal:** the other four fusion streams + maintenance events that reset curves.
 - **Steps:** spares inventory with lead times; crew trades/shifts/capacity; 7-day sortie schedule with priorities; maintenance records consistent with failures (a fix consumes a spare + crew-hours and resets the curve).
 - **Files:** extend `gen/generate.py` ; `gen/seed.py` . **Deps:** T3. **Skill:** `data-eng` .
 - **Done: determinism test passes** — same seed → identical SHA-256 of DB; seed recorded in `metrics.json` .
-

PHASE B — ML EVIDENCE (hours 8–14)

T5. Feature store (single code path)

- **Goal:** windowed features usable identically in train and inference.
- **Steps:** rolling mean/std/min/max/slope over 10- and 30-cycle windows per sensor + cycle + `cycles_since_overhaul` + one-hots; one function `features/build.py` , no separate inference copy.
- **Files:** `features/build.py` . **Deps:** T4. **Skill:** `ml-eng` .
- **Done:** unit test — feature frame for a known tail matches hand-computed window stats on a toy series.

T6. RUL model train + eval (THE evidence task)

- **Goal:** HistGradientBoostingRegressor per component class; RMSE/MAE vs naive baseline.
- **Steps:** target RUL capped at 125 h; early stopping; **split BY TAIL**; `models/eval.py` prints RMSE/MAE vs mean-RUL baseline; save `model.pkl` + `metrics.json` per class.
- **Files:** `models/train.py` , `models/eval.py` , `models/registry.py` . **Deps:** T5. **Skill:** `ml-eng` .
- **Done:** `python models/eval.py` prints a table (per class: RMSE, MAE, baseline RMSE, baseline MAE); `metrics.json` written. **This printout is judge evidence — do not proceed to UI until it exists.**

T7. Prediction intervals + inference service

- **Goal:** `ci_low/ci_high` per prediction; batch inference writing `prediction` rows.
 - **Steps:** quantile GBM or $\pm 1.5\times$ validation MAE (document choice); `models/infer.py` writes predictions with `run_id`.
 - **Files:** `models/infer.py` . **Deps:** T6. **Skill:** `ml-eng` .
 - **Done:** every component has a current prediction row with intervals; spot-check intervals widen on noisier units.
-

PHASE C — DECISION LOGIC (hours 14–22)

T8. Availability aggregator (pure function)

- **Goal:** state → per-tail MC/PMC/NMC, 7-day MC forecast, unfilled sorties (§4.4).
- **Steps:** implement R1/R2/R3 rules reading thresholds from config; `core/aggregator.py` takes a state dict, returns results; no global state.
- **Files:** `core/aggregator.py` , `core/state.py` (snapshot load/clone). **Deps:** T4, T7. **Skill:** `backend-eng` .
- **Done:** unit test — hand-built state (one tail RUL<24h, one tail no-spares, one tail no-crew) yields exactly the expected NMC set and drivers.

T9. What-if engine

- **Goal:** clone → lever patch → recompute → diff → store scenario (§4.5).
- **Steps:** lever types expedite_spares / reassign_crew / cannibalize as pure patches; `core/whatif.py` ; latency check.
- **Files:** `core/whatif.py` . **Deps:** T8. **Skill:** `backend-eng` .
- **Done:** **recompute < 200 ms** on the demo fleet; base state byte-identical after a what-if run; scenario row persisted.

T10. NMC-driver ranker

- **Goal:** binding constraint per tail, fleet-impact score, top-5 (§4.6).
- **Steps:** `core/drivers.py` ; score = tails × avg priority × days grounded.
- **Files:** `core/drivers.py` . **Deps:** T8. **Skill:** `backend-eng` .
- **Done:** unit test on hand-built state returns the known top driver with the known score.

T11. Cannibalization advisor + guardrails + debt ledger entries

- **Goal:** donor proposal with hard guardrails (§4.7); debt entry on approval.
- **Steps:** `core/cannibal.py` ; guardrail checks (lower priority, already NMC, CANN-rate warn); net-MC effect calc; write debt entry.
- **Files:** `core/cannibal.py` . **Deps:** T8. **Skill:** `backend-eng` .
- **Done:** **unit tests refuse** equal/higher-priority donor and MC donor; approval writes debt-ledger entry + audit entry.

T12. Crew 1-day greedy scheduler + audit log

- **Goal:** F9 scheduler + F8 hash-chained audit.
 - **Steps:** `core/scheduler.py` (greedy by priority × driver score, never over-allocates); `core/audit.py` (append with prev_hash/row_hash, verify function).
 - **Files:** `core/scheduler.py` , `core/audit.py` . **Deps:** T8, T10. **Skill:** `backend-eng` .
 - **Done:** scheduler unit test (capacity never exceeded; overflow → backlog with reason); **tamper test** (edit a row → verify fails).
-

PHASE D — API (hours 22–26)

T13. FastAPI endpoints

- **Goal:** all routes from the API sketch in `03-architecture-techstack.md`.
 - **Steps:** thin routes calling core/; *Pydantic schemas*; every response includes `"data_class": "synthetic"`; `/api/whatif` accepts lever JSON; `/api/cannibalization/advice?recipient=`.
 - **Files:** `api/routes/*.py`, `api/schemas.py`. **Deps:** T8–T12. **Skill:** backend-eng.
 - **Done: API contract tests** — all endpoints 200 on the demo DB; every response body contains `data_class=synthetic`.
-

PHASE E — DASHBOARD (hours 26–33)

T14. Fleet board (Canvas 2D)

- **Goal:** the commander's morning screen — tail cards with status colors, MC/PMC/NMC counts, **SYNTHETIC DATA banner**.
- **Steps:** `web/index.html`, `web/app.js`, `web/fleet.js` (Canvas card grid); banner component reused on every screen.
- **Files:** `web/*`. **Deps:** T13. **Skill:** frontend-eng.
- **Done:** board renders 40 tails from `/api/fleet/status`; banner visible; **banner presence test** (DOM check).

T15. Tail drill-down + explainability panel

- **Goal:** click a tail → components, RULs with intervals, top contributing features + template text.
- **Steps:** `web/tail.js`; explainability from sklearn contributions; template text uses only prediction-row numbers.
- **Files:** `web/tail.js`. **Deps:** T14. **Skill:** frontend-eng.
- **Done:** drill-down shows RUL + CI + top-3 features for any component; text contains no invented numbers.

T16. Forecast chart + what-if panel

- **Goal:** 7-day MC curve + unfilled sorties; lever controls with live before/after delta.
- **Steps:** `web/forecast.js` (Canvas line chart), `web/whatif.js` (lever form → POST `/api/whatif` → side-by-side delta).
- **Files:** `web/forecast.js`, `web/whatif.js`. **Deps:** T14. **Skill:** frontend-eng.
- **Done:** applying a lever updates the delta panel with server-computed numbers in < 1 s round-trip.

T17. Cannibalization advisor UI + audit log tab

- **Goal:** donor proposal view with guardrail results; audit trail view.
- **Steps:** `web/cannibal.js`, `web/audit.js`.
- **Files:** `web/cannibal.js`, `web/audit.js`. **Deps:** T16. **Skill:** `frontend-eng`.
- **Done:** advisor shows donor, net effect, and which guardrails passed; audit tab lists actions reverse-chron.

PHASE F — DEMO & EVIDENCE (hours 33–36)

T18. Scripted demo scenario + honesty pass

- **Goal:** a named 7-day scenario with a guaranteed visible MC delta; full honesty sweep.
- **Steps:** pick seed + lever set (e.g., expedite 2 parts + 1 cannibalization) that yields a clear before/after; rehearse the 5-minute script (`06-demo-and-judge-qa.md`); verify SYNTHETIC banner on every screen, synthetic tail numbers, `data_class` on every API response.
- **Files:** `demo/scenario.md` (seed, levers, expected deltas — **expected deltas are recomputed live, never hard-coded into the UI**). **Deps:** T14–T17. **Skill:** `qa-eng` + `frontend-eng`.
- **Done:** dry run completes in ≤ 5 min; every honesty check in the sweep passes.

T19. Evidence bundle + final test run

- **Goal:** everything a judge can ask for, printed and passing.
- **Steps:** run full pytest; capture `models/eval.py` printout; capture what-if latency; capture generator SHA-256 determinism proof; save to `evidence/` (printouts + `metrics.json` copy); final `README.md` run instructions (one command to launch).
- **Files:** `evidence/*`. **Deps:** T18, all tests. **Skill:** `qa-eng`.
- **Done:** `pytest` green; evidence folder contains: eval printout, latency number, determinism SHA-256, API contract results, banner test result. **Nothing in the deck claims a number not found in `evidence/`.**

Test/evidence checklist (maps to T-tasks)

Check	Task	Type
Generator determinism (same seed → same SHA-256)	T4	automated
Feature correctness vs hand-computed windows	T5	unit
RMSE/MAE vs naive baseline (judge evidence)	T6	eval script printout

Check	Task	Type
Aggregator hand-built state → expected NMC set	T8	unit
What-if < 200 ms; base state unmutated	T9	perf + unit
Driver ranking known-answer	T10	unit
Cannibalization guardrail refusals	T11	unit
Scheduler capacity never exceeded	T12	unit
Audit tamper → verify fails	T12	unit
Every DB row synthetic=1	T13	automated
Every API response data_class=synthetic	T13	contract
UI banner on every screen	T14	DOM test

P1 stretch insertion points (only if ahead of schedule)

After T16: F15 daily brief (template text, LOW). After T17: F11 debt-ledger UI, F12 watchlist (both LOW). F13/F14/F16 only if Phase E finished early — F14 needs the read-only replay path. P2 items are never started.